Understand Why Raw Types Are Unsafe (Interview Notes)

What are Raw Types?

A **raw type** is a generic class or interface used **without specifying its type parameter**.

Example:

```
ArrayList list = new ArrayList();
```

Instead of:

```
ArrayList list = new ArrayList<>();
```

The first example is called a **raw type**.

Why Are Raw Types Unsafe?

Without generics, Java treats elements as **Object**.

This means you can accidentally store different types in the same collection.

```
ArrayList list = new ArrayList();

list.add("Java");
list.add(100);
list.add(true);
```

The compiler allows all of them.

Later,

```
String s = (String) list.get(1);
```

Output

```
ClassCastException
```

Because 100 is an Integer, not a String.

How Generics Solve This

```
ArrayList list = new ArrayList<>();

list.add("Java");
list.add("Spring");
```

Trying to add an integer:

```
list.add(100);
```

Produces a **compile-time error**, preventing the bug before the program runs.

Raw Type vs Generic Type

Raw Type	Generic Type
ArrayList list	ArrayList list
Stores any object	Stores only specified type
Requires casting	No explicit casting
Less type-safe	Compile-time type safety
May cause ClassCastException	Prevents many runtime errors

Important Interview Points ⭐

- A **raw type** is a generic class used **without a type parameter**.

- Raw types disable **compile-time type checking**.
- They allow inserting **mixed object types** into the same collection.
- Retrieving elements often requires **explicit casting**, increasing the risk of ClassCastException.
- Generics provide **type safety** by catching type mismatches at compile time.
- Raw types exist mainly for **backward compatibility** with code written before Java 5.
- Modern Java code should **avoid raw types** unless interacting with legacy APIs.

Tricky Interview Questions

Q1. What is a raw type?

Answer:

A generic class or interface used without specifying its type parameter.

Example:

```
List list = new ArrayList();
```

Q2. Why are raw types considered unsafe?

Answer:

Because they disable compile-time type checking and can lead to ClassCastException at runtime.

Q3. Why do raw types still exist in Java?

Answer:

For **backward compatibility** with legacy code written before generics were introduced in Java 5.

Q4. Which is better?

```
List list = new ArrayList();
```

or

```
List list = new ArrayList<>();
```

Answer:

List because it provides compile-time type safety.

Q5. Do raw types generate compiler warnings?

Answer: Yes.

Java issues **"unchecked" warnings**, indicating that type safety cannot be guaranteed.

Q6. Can raw types cause runtime exceptions?

Answer: Yes.

Most commonly, they can lead to a **ClassCastException** when retrieving elements.

Interview Summary ⭐

- **Raw types are generic classes used without specifying a type parameter.**
- They **disable compile-time type checking**, allowing mixed object types.
- This increases the risk of **ClassCastException** at runtime.
- Generics eliminate these risks by enforcing **type safety** during compilation.
- Raw types should be used **only for compatibility with legacy code**, not in new applications.

> **Interview One-Liner:**
>
> **"Raw types bypass Java's generic type checking, making collections unsafe by allowing mixed data types and increasing the risk of ClassCastException at runtime."**